HealTech PerDeCT Wave Analyzer
Architecture & App Process Outline

1. Complete Code Structure

Core Application (Backend web server & API)
* **`app.py`**
  * The main FastAPI application.
  * Exposes the `/analyze` endpoint which receives raw `.wav` uploads from the frontend dashboard.
  * Handles the entire signal processing chain asynchronously, generates the diagnostic report text, downsamples output for UI rendering, and creates the frequency heatmap (`spec_...png`) and CSV data downloads.

Digital Signal Processing Engine
* **`dsp_utils.py`**
  * The mathematical foundation of the application.
  * `wavelet_denoise()`: Drops standard electronic noise without destroying heartbeat morphology.
  * `butter_bp()`: Butterworth bandpass filter to isolate blood flow Doppler shifts.
  * `kasai_fd()`: Uses the Kasai lag-1 autocorrelation method to detect instantaneous frequency shifts in the complex audio signal.
  * `velocity_from_fd()`: Converts raw frequency shift (Hz) into physical blood velocity (m/s).
  * `nlms_adaptive_filter()`: (Used in offline pipelines) Adaptive noise cancellation using accelerometer data.

Machine Learning & Verification
* **`drug_labels.py`**
  * The ground-truth registry containing the known experimental states of the pig trials (Baseline, Esmolol, Dobutamine).
* **`train_model.py`**
  * Offline training script that generates two models from the `data/` folder:
    1. **`hypovolemia_model.pkl`**: An unsupervised `IsolationForest` detecting anomalous dropping baseline patterns indicative of hemorrhage.
    2. **`drug_condition_model.pkl`**: A supervised `RandomForestClassifier` trained on rolling waveform features labeled by `drug_labels.py` to recognize specific drug influence (like beta-blockers or inotropes).
* **`verify_model.py`**
  * A diagnostic testing script that rapidly sweeps through all recorded `.wav` files, performs the DSP chain natively, and runs the output through both ML models to print a graded performance report.

Testing & Offline Pipelines
* **`test_integration.py`**
  * A fast script that hits the live `localhost:8000/analyze` API to ensure the response JSON parses correctly and the confidence/accuracy metrics exist.
* **`pipeline.py`**
  * Experimental offline task framework meant for motion cancellation and linear regression Medistim-calibration.

---

2. Explanation of the App Process

When a raw doppler ultrasound file is uploaded to the dashboard, the backend triggers the following fully automated sequence:

Step 1: Ingestion & Pre-cleaning (`app.py` & `dsp_utils.py`)
The system reads the I/Q stereo `.wav` file into a numpy array framework. The data goes through a Daubechies Level-3 Discrete Wavelet Transform to drop minor random interference spikes, followed by a 1,500 Hz to 10,000 Hz Bandpass filter to completely isolate frequencies corresponding to actual blood cell movement.

Step 2: Frequency & Velocity Extraction (`dsp_utils.py`)
The cleaned audio signal is fed into the Kasai autocorrelator. By looking at the phase shift between consecutive audio samples, the Kasai algorithm determines exactly what the Doppler shift frequency was at that split second. This frequency is then put through the Doppler equation to convert it directly to physical blood flow velocity (m/s).

Step 3: Cardiac Output (CO) Calculation (`app.py`)
Velocity is multiplied by the estimated cross-sectional area of the aorta (and converted to Liters per Minute) to approximate real-time Cardiac Output (CO). The resulting jagged output is heavily smoothed using a 15 Hz lowpass filter to create a clean, fluid wave that looks like a standard ECG or pressure trace.

Step 4: Feature Extraction & Machine Learning
The newly created CO signal track is broken down into small rolling "windows". For every window, the code calculates:
* `mean` (Average baseline output)
* `ptp` (Peak-to-Peak: how high the pulse jumps)
* `std` (Variance)

These five features per window are simultaneously fed into two live ML models:
1. **The IsolationForest** flags how many windows represent abnormal, collapsing behavior in the subject's cardiac condition (warning for shock or Arrythmia).
2. **The RandomForest Classifier** looks at the features and attempts to guess what pharmacological drug state the subject is currently in based on millions of training samples from previous Esmolol/Dobutamine trials.

Step 5: Report Generation
The system detects the individual physical "beats" in the wave to calculate Heart Rate (BPM) and Heart Rate Variability (HRV). It synthesizes all the ML telemetry, beat data, and confidence statistics into a human-readable **Diagnostic Report**.

Step 6: Presentation
The system downsamples the millions of data points into 4,000 visual data points for high-performance React

plotting. It bundles the Report, the downsampled plot lines, a generated Heatmap image (`spec_*.png`), and a full CSV telemetry payload back to the browser in a lightning-fast JSON response.